MODULE 2 · LECTURE 7

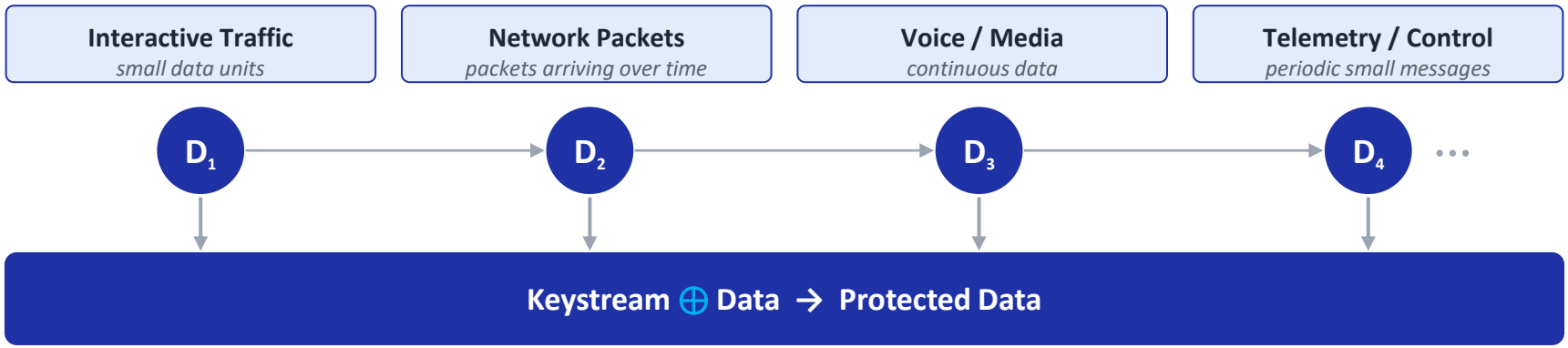
CS 6530 Applied Cryptography

Modern Stream Ciphers: Salsa20 and ChaCha20

Module 2 – Symmetric Cryptography, Hashes and Authentication

ENGINEERING MOTIVATION

Why Stream-Oriented Encryption?



Engineering Insight

Stream-oriented encryption is an operational model, not simply encryption for audio/video "streams." Network traffic, telemetry and interactive messages can all benefit from incremental processing.

Compact Operational Model

Model: $C = P \oplus KS$
(plaintext processed incrementally as it becomes available)

Think Like an Engineer

If data is continuously generated, should encryption wait for the complete message? Expected: No — protection should proceed as data becomes available.

Stream-oriented encryption protects data incrementally as it becomes available, without requiring block padding.

CONNECT TO WHAT WE KNOW

From CTR Mode to a Purpose-Designed Stream Cipher

AES-CTR

Counter Block (i)

↓

AES + Key (K)

↓

Keystream Block (KS_i)

$$P_i \oplus KS_i = C_i$$

Block cipher used as a keystream generator

ChaCha20

Key (K) + Nonce (N) + Counter (i)

↓

ChaCha20

↓

Keystream Block (KS_i)

$$P_i \oplus KS_i = C_i$$

Cipher designed directly for keystream generation

Engineering Insight

CTR already gave us stream-like encryption. ChaCha20 changes the keystream generator, while retaining the XOR-based encryption model.

Compact Operational Model

AES-CTR: $KS_i = E_k(\text{CounterBlock}_i)$, $C_i = P_i \oplus KS_i$

ChaCha20: $KS_i = \text{ChaCha20Block}(K,N,i)$, $C_i = P_i \oplus KS_i$

Think Like an Engineer

In CTR mode, what does AES actually encrypt?

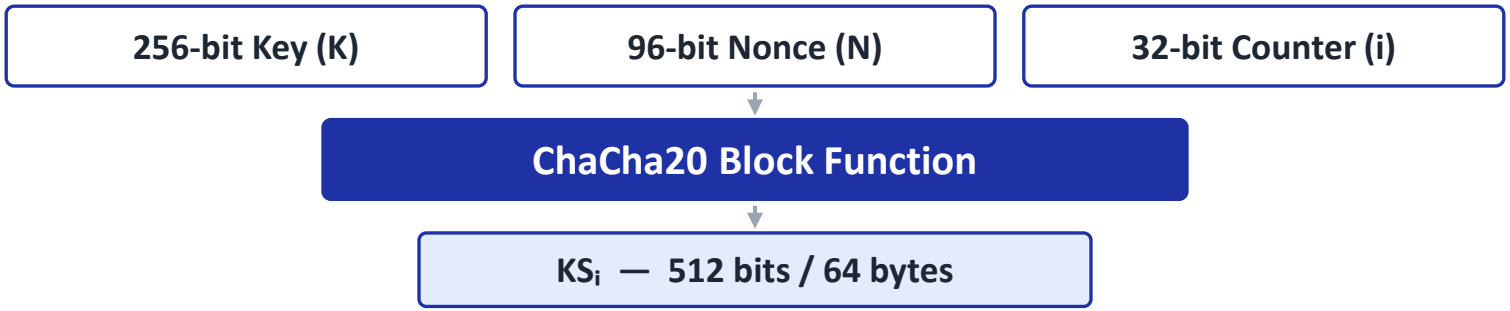
— The counter block, not the plaintext directly.

CTR turns a block cipher into a keystream generator; ChaCha20 is designed directly for that role.

OPERATIONAL MODEL

The Keystream Model: Key, Nonce and Counter

Worked Example — IETF ChaCha20



KEY	NONCE	COUNTER
Secret	*Distinguishes encryptions*	*Distinguishes successive blocks*

Engineering Insight

A stream cipher need not generate one bit or byte at a time. ChaCha20 generates 64-byte blocks that collectively form the keystream.

Compact Operational Model

Model: $KS_i = F(K, N, i)$

ChaCha20: $(K,N,i) \rightarrow KS_i$, $|KS_i| = 512$ bits = 64 bytes

Use: $C = P \oplus KS$, $P = C \oplus KS$

Think Like an Engineer

If the nonce stays fixed within an encryption, what makes successive keystream blocks different? — The counter.

The key provides secrecy, the nonce distinguishes encryptions, and the counter distinguishes successive keystream blocks.

SECURITY RULE — RECALL

Keystreams Must Not Repeat

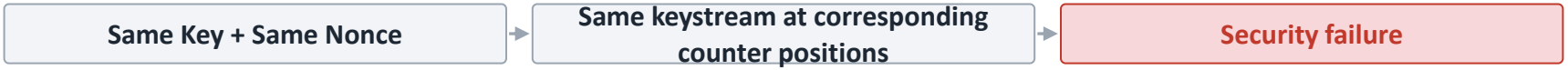
What happens if the same keystream encrypts two different plaintexts?



$$C_1 \oplus C_2 = (P_1 \oplus KS) \oplus (P_2 \oplus KS)$$

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

Keystream disappears → relationship between plaintexts is exposed



Engineering Insight

Keystream reuse does not automatically reveal both plaintexts, but it removes the keystream from their relationship. Known plaintext, predictable formats, redundancy or guesses may then be exploited.

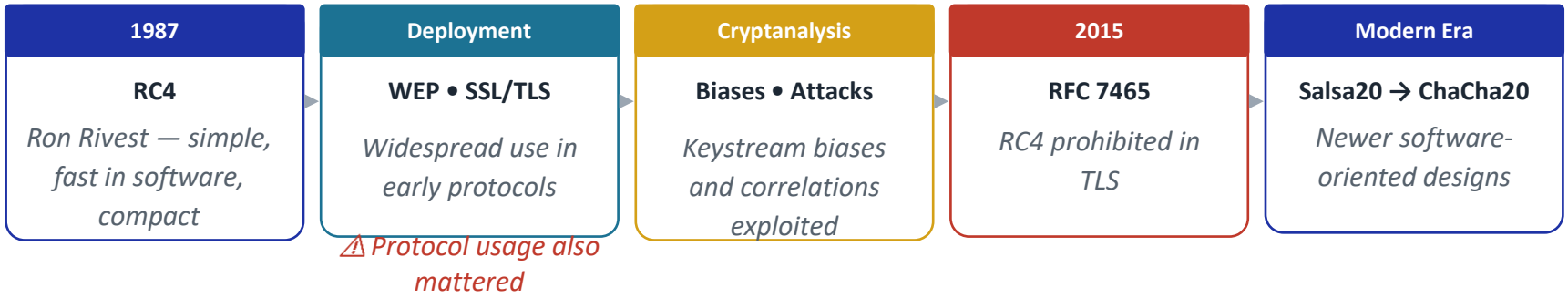
? Think Like an Engineer

We saw this with CTR — what information does keystream reuse expose? (Allow a response before revealing P1 XOR P2.)

Stream encryption remains secure only when the same keystream is never reused for different plaintexts.

LESSONS FROM HISTORY

From RC4 to Modern Stream Ciphers



Key → Internal State → KS $C = P \oplus KS$

Engineering Insight

Cryptographic agility matters — systems must be able to retire algorithms when new cryptanalysis changes their security assessment. Primitive weakness and protocol misuse can compound each other.

Compact Operational Model

Model: Key → Internal State → KS, $C = P \oplus KS$
Note: No KSA / PRGA internals shown (out of scope for this lecture)

? Think Like an Engineer

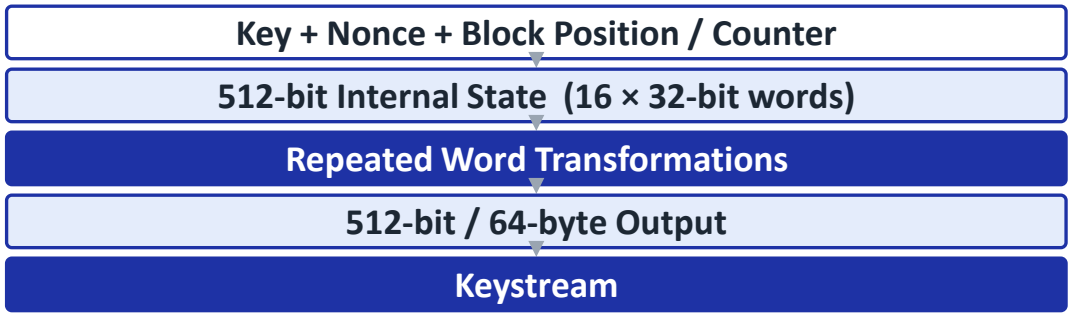
If RC4 was simple, fast and widely deployed, why wasn't that enough?

RC4's history shows why deployed cryptography must evolve when cryptanalysis exposes weaknesses.

MODERN DESIGN DIRECTION

Salsa20: A Software-Oriented Stream Cipher

Daniel J. Bernstein — Salsa20 — eSTREAM software portfolio



Operation Family
+ Modular Addition, ↻ Rotation, ⊕ XOR

Salsa20 design philosophy
↓ *Refinement* ↓
ChaCha

Can strong cryptographic mixing be built from simple processor operations without S-boxes?

Engineering Insight

Salsa20 was designed around operations efficiently supported by general-purpose processors, avoiding S-box-based substitution — but this does not mean it is universally faster than AES.

Compact Operational Model

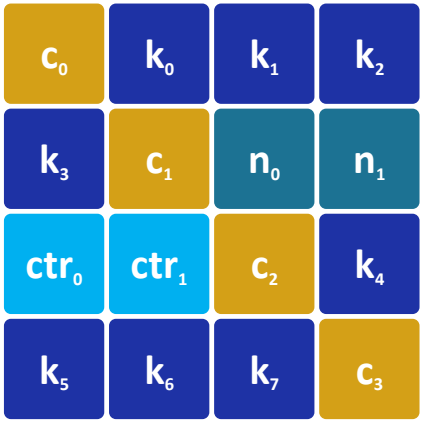
State: 16 × 32-bit words = 512 bits
Model: State → (repeated word operations) → 512-bit output
Teaching focus: Salsa20/20 — 20 rounds

Think Like an Engineer

Can strong cryptographic mixing be built using only simple processor-friendly operations? Slide 7 answers this.

Salsa20 showed how strong stream encryption could be built from simple, software-friendly word operations.

Inside Salsa20: The 4×4 State



CONSTANTS
4 × 32-bit words

KEY
8 × 32-bit words

NONCE
2 × 32-bit words

COUNTER
2 × 32-bit words

128 (Constants) + 256 (Key) + 64 (Nonce) + 64 (Counter) = 512 bits = 16 × 32-bit words

Engineering Insight

The state layout is not merely storage. The positions determine which words interact as Salsa20 applies its round transformations.

Compact Operational Model

Model: X = State(Constants, K, N, Counter)
Size: 16 × 32 = 512 bits
With: K = 256 bits, N = 64 bits, Counter = 64 bits

Think Like an Engineer

We have placed the key, nonce and counter into the state — but nothing has been encrypted yet. What must happen next? — The state must be thoroughly mixed.

Salsa20 begins by arranging constants, key, nonce and counter into a 512-bit working state.

ARX IN ACTION

Salsa20 Quarter Round: How Four Words Are Mixed

y_0	y_1	y_2	y_3
$y_0 + y_3$ → ROTL 7 →	⊕ y_1 →		z_1
$z_1 + y_0$ → ROTL 9 →	⊕ y_2 →		z_2
$z_2 + z_1$ → ROTL 13 →	⊕ y_3 →		z_3
$z_3 + z_2$ → ROTL 18 →	⊕ y_0 →		z_0

Rotation distances: 7 • 9 • 13 • 18

ADD *(carry-dependent)* **ROTATE** *(no bits lost)* **XOR** *(combines words)*

Engineering Insight

AES uses substitution + linear mixing; Salsa20 uses addition + rotation + XOR. Different mechanisms, common objective: rapidly spread dependencies. Note: Rotate ≠ Shift — rotation discards no bits.

Compact Operational Model

Model: $QR(y_0,y_1,y_2,y_3) \rightarrow (z_0,z_1,z_2,z_3)$
Using: $+ \lll \oplus \ (+ \bmod 2^{32})$
Rotations: 7, 9, 13, 18

Think Like an Engineer

How can such simple processor operations produce strong cryptographic mixing? — Each operation contributes, and repeated dependencies spread the changes.

Salsa20's quarter round uses modular addition, rotation and XOR to progressively mix four 32-bit words.

FROM MIXING TO KEYSTREAM

From Quarter Rounds to the Salsa20 Keystream

feed-forward — original X bypasses the core

Initial State X → **Column Round → Row Round = 1 Double Round × 10 → 20-Round Salsa20 Core → Y** → **KS = X + Y mod 2^{32}** → **Serialize → 512 bits 64-byte Keystream Block**

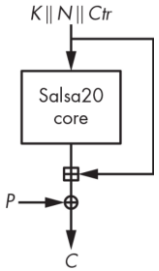
State A — Counter …00000000
State B — Counter …00000001
(only 1 bit differs)

20 Rounds →

Dramatically different 512-bit states
Diffusion / Avalanche in Action (illustrative — see Serious Cryptography, Listings 5-3 / 5-4)

Engineering Insight

The Salsa20 core is built from invertible ARX operations. Feed-forward — X + Salsa20Core(X) — prevents the keystream from simply exposing that invertible core's output.



Compact Operational Model

Model: Y = Salsa20Core(X)
Keystream: KS = X + Y mod 2^{32}
Then: serialize → 64-byte keystream block

Think Like an Engineer

If ARX operations are reversible, why can't an attacker reverse the 20 rounds to recover the key? — The exposed keystream is X+Y, not the core output Y alone.

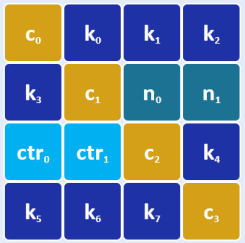
Salsa20 combines repeated ARX mixing with feed-forward of the initial state to produce each 64-byte keystream block.

DESIGN EVOLUTION

Why ChaCha20 After Salsa20?

Salsa20 already provides an elegant ARX stream cipher. Why design ChaCha20?

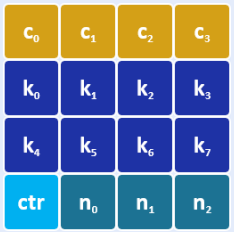
Salsa20 — Retained Foundations



ARX • 7 • 9 • 13 • 18 • Column + Row

REFINEMENT

ChaCha20 — What Changes?



ARX • 16 • 12 • 8 • 7 • Column + Diagonal

Shared foundation: 512-bit State • 32-bit Words • ARX • 20 Rounds • Feed-Forward • 64-byte Output

Engineering Insight

Cryptographic evolution is not limited to replacing failed algorithms. A sound design may be refined to improve diffusion, security margin, or implementation characteristics while retaining successful ideas.

Compact Operational Model

Evolution: Salsa20 → (redesign mixing, retain ARX philosophy) → ChaCha20
Shared: 16 × 32 = 512-bit state
Changed: QR(Salsa20) ≠ QR(ChaCha20)

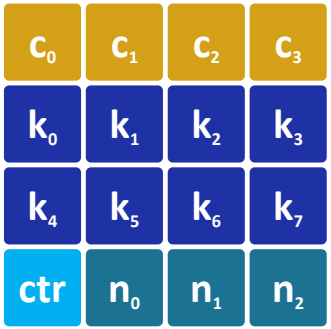
? Think Like an Engineer

If Salsa20 was not broken, why design another cipher? — A cipher need not be broken before engineers seek stronger diffusion or simpler structure.

ChaCha20 refines Salsa20's ARX design to achieve stronger diffusion per round while preserving software-oriented simplicity.

CHACHA20 STATE

Inside the ChaCha20 State



ROW 0 — Constants
4 × 32 bits = 128 bits

ROWS 1–2 — Key
8 × 32 bits = 256 bits

ROW 3 — Counter + Nonce
32 + 96 = 128 bits

128 (Constants) + 256 (Key) + 32 (Counter) + 96 (Nonce) = 512 bits — words are little-endian

Engineering Insight

The same broad ingredients remain, but their arrangement changes. This IETF ChaCha20 state uses a 256-bit key, 32-bit counter, and 96-bit nonce, prepared for the column/diagonal round schedule.

Compact Operational Model

State: X = [C C C C / K K K K / K K K K / Ctr N N N]
Words: 4C + 8K + 1Ctr + 3N = 16 words
Size: 16 × 32 = 512 bits

Think Like an Engineer

Compare the Salsa20 and ChaCha20 states. What structural change do you notice? — ChaCha20 groups constants, key, and counter/nonce into regular rows.

IETF ChaCha20 organizes constants, a 256-bit key, a 32-bit counter and a 96-bit nonce into a regular 512-bit state.

The ChaCha20 Quarter Round

a	b	c	d	
a = a + b	d = d ⊕ a	ROTL by 16		*Step 1*
c = c + d	b = b ⊕ c	ROTL by 12		*Step 2*
a = a + b	d = d ⊕ a	ROTL by 8		*Step 3*
c = c + d	b = b ⊕ c	ROTL by 7		*Step 4*

ChaCha20 rotations: 16 • 12 • 8 • 7 *Salsa20 rotations: 7 • 9 • 13 • 18 — same ARX vocabulary, redesigned dependency structure*

Engineering Insight

Each operation is simple — ADD, XOR, ROTATE — but the important design feature is their composition and dependency structure. Repeated quarter rounds spread a small change across more words and bit positions.

Quarter Round – Compact Notation

QR(a,b,c,d): a=a+b; d=(d⊕a)⋘16; c=c+d; b=(b⊕c)⋘12; a=a+b; d=(d⊕a)⋘8; c=c+d; b=(b⊕c)⋘7

Think Like an Engineer

ChaCha20 still uses only Add, Rotate, and XOR. So where does the stronger mixing come from? — The redesigned dependencies and rotation choices.

ChaCha20's quarter round reorganizes ARX dependencies so changes rapidly propagate across all four participating words.

WHOLE-STATE MIXING

From Quarter Rounds to ChaCha20

COLUMN ROUND



QR(0,4,8,12) • QR(1,5,9,13) • QR(2,6,10,14) • QR(3,7,11,15)

then →

DIAGONAL ROUND



QR(0,5,10,15) • QR(1,6,11,12) • QR(2,7,8,13) • QR(3,4,9,14)

Column Round + Diagonal Round = 1 Double Round • 10 Double Rounds = 20 Rounds

Salsa20: Column → Row | ChaCha20: Column → Diagonal — *different schedules, same objective: spread dependencies*

Engineering Insight

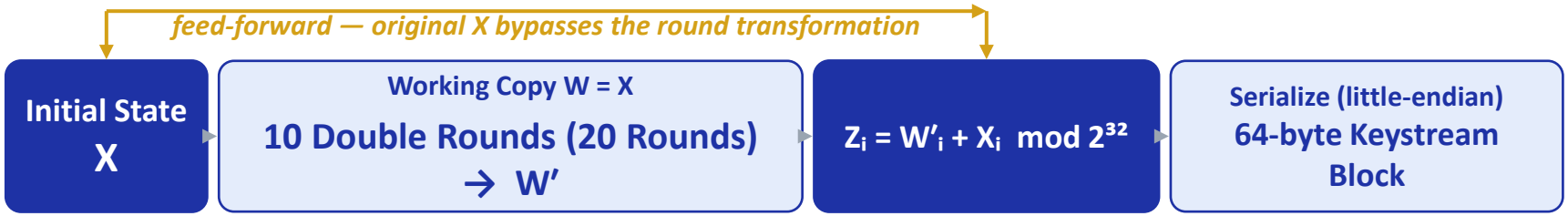
The security-relevant idea is not merely column versus diagonal — it is changing which words interact as rounds progress. That changing connectivity drives diffusion across the 512-bit state.

? Think Like an Engineer

Why not repeatedly apply quarter rounds to the same four groups of words? — Changing the groupings lets dependencies spread throughout the entire state.

ChaCha20 alternates column and diagonal quarter rounds so dependencies rapidly spread across the entire 512-bit state.

From State to a ChaCha20 Keystream Block



$C = P \oplus KS$

$(K,N,i) \rightarrow KS_i$, $(K,N,i+1) \rightarrow KS_{i+1}$
2^{32} counter values × 64 bytes ≈ 256 GiB per nonce

Engineering Insight

The exposed keystream is not simply the transformed working state — it is Transformed State + Original State. This feed-forward prevents direct inversion of the round output to recover the secret state.

Think Like an Engineer

We saw this in Salsa20 — what must happen before the transformed state becomes the exposed keystream? — The original state must be added back.

ChaCha20 adds the original state to the 20-round result, then serializes it into a 64-byte keystream block.

SOFTWARE ENGINEERING

Why ChaCha20 Works Well in Software

ADD + XOR + ROTATE

1 — Processor-Friendly Operations

32-bit addition, XOR, fixed rotations map naturally to general-purpose processor operations.

2 — No Secret-Indexed Tables

No S-box lookup inherent to the design → facilitates constant-time implementations. (Implementation still

3 — Independent Keystream Blocks

$KS_i = F(K,N,i)$ — different counter values allow independent generation → parallel/SIMD, random access.

4 — No Padding

$C = P \oplus KS$ — only the required keystream bytes are used; no block-padding scheme required.

CPU A	Same cryptographic choice	CPU B
AES acceleration ✓ → very efficient	*on every platform? → Not necessarily.*	*AES acceleration ✗ → ChaCha20 attractive*

Engineering Insight

Algorithm design helps implementation security — it does not guarantee it. Actual performance and side-channel resistance still depend on the implementation, compiler, processor, and system.

Compact Operational Model

Independence: $KS_i = F(K,N,i) \implies$ independent blocks

Encryption: $C = P \oplus KS$

Core ops: $+ \bmod 2^{32}$, $\oplus$, ROTL

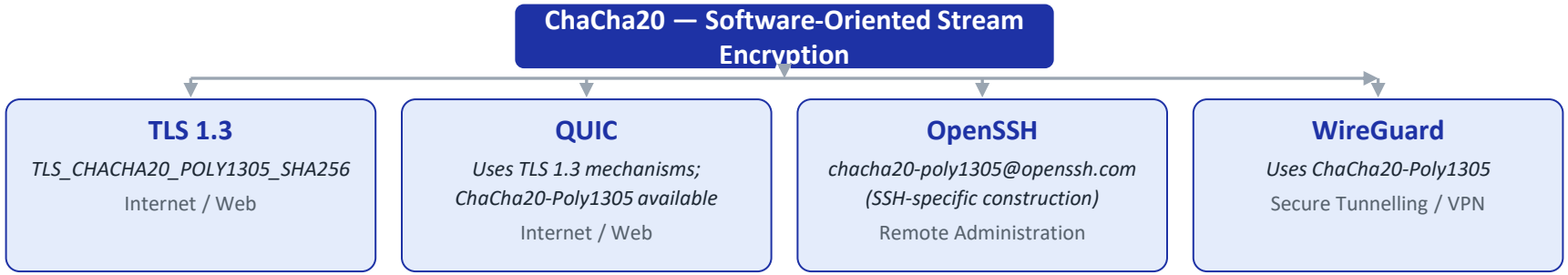
Think Like an Engineer

One processor has AES hardware acceleration; another does not. Would performance stay the same? — No: platform capabilities shape cryptographic choices.

ChaCha20 combines simple ARX operations with software-friendly, padding-free, independently generated keystream blocks.

FROM CIPHER TO SYSTEM

ChaCha20 in Real Systems



Notice: deployed systems commonly say "ChaCha20-Poly1305," not simply "ChaCha20."

Why? → Next Lecture

Engineering Insight

A cryptographic primitive is ultimately useful only when it can be implemented and deployed safely in real systems.
Cryptographic strength + implementation characteristics + platform capabilities → deployment choice.

Compact Operational Model

Studied: ChaCha20
Deployed as: ChaCha20-Poly1305
Examples: TLS 1.3 • QUIC • OpenSSH • WireGuard

Think Like an Engineer

Why might a protocol standard provide more than one strong cryptographic algorithm rather than mandate one for every platform? — The target platform matters.

ChaCha20's software-oriented design has made it a practical cryptographic component across Internet, remote-access and VPN systems.

Beyond ARX: SNOW 3G and ZUC

Do all modern stream ciphers use ARX (Addition • Rotation • XOR)? **No.**

SALSA20 / CHACHA20

Word State
↓ ARX Mixing ↓
Keystream

SNOW 3G

LFSR
↓ Nonlinear FSM ↓
Keystream

3GPP / Cellular • UEA2 (confidentiality) • UIA2 (integrity)

ZUC

LFSR
↓ Bit Reorg. + Nonlinear Fn ↓
Keystream

3GPP / Cellular • 128-EEA3 (confidentiality) • 128-EIA3 (integrity)

TAKE NOTE — AWARENESS LEVEL: *internal operation of SNOW 3G / ZUC is outside this lecture's scope. Deeper study is optional.*

Engineering Insight

"Stream cipher" describes a cryptographic category — it does not prescribe one internal architecture. Salsa20/ChaCha20 use ARX; SNOW 3G/ZUC use telecom-oriented evolving-state designs. Common objective: a secure keystream.

Compact Operational Model

Broadest model: Key + Init Inputs + State → Keystream Gen. → Keystream
Salsa20 / ChaCha20: ARX architecture
SNOW 3G / ZUC: evolving-state architectures

Think Like an Engineer

Why might telecom standards use different stream-cipher architectures than Internet cryptography? — Standards history and target hardware matter.

Modern stream ciphers can use very different internal designs; SNOW 3G and ZUC illustrate the telecom-oriented design space beyond ARX.

ENGINEERING TAKEAWAYS

Modern Stream Ciphers: From Keystream to Deployment



"Principles endure. Technologies evolve. Engineering bridges the two."

RC4 → Salsa20 → ChaCha20 *alongside* **SNOW 3G • ZUC**

We can generate a secure, unique keystream and protect confidentiality. Is confidentiality alone enough? **NEXT: AUTHENTICATED ENCRYPTION**

Compact Operational Model

Model: (K,N,Counter) → KS_i, $P_i \oplus KS_i = C_i$
Critical rule: never reuse the keystream under the same key
Chain: Requirement → Construction → Inputs → Impl. → Deployment

Think Like an Engineer

Before deploying: what services are needed, which construction is standardized, how are keys/nonces managed, and what does the platform support?

Further Reading / References

Core Reading

Aumasson, Serious Cryptography

Paar & Pelzl, Understanding Cryptography

Specification

RFC 8439 — ChaCha20 and Poly1305 for IETF Protocols

Optional Telecom Reading

3GPP / ETSI-SAGE specifications (SNOW 3G, ZUC)

Secure stream encryption depends not only on the cipher, but on unique keystream generation, correct initialization, platform-aware implementation, and disciplined deployment.